

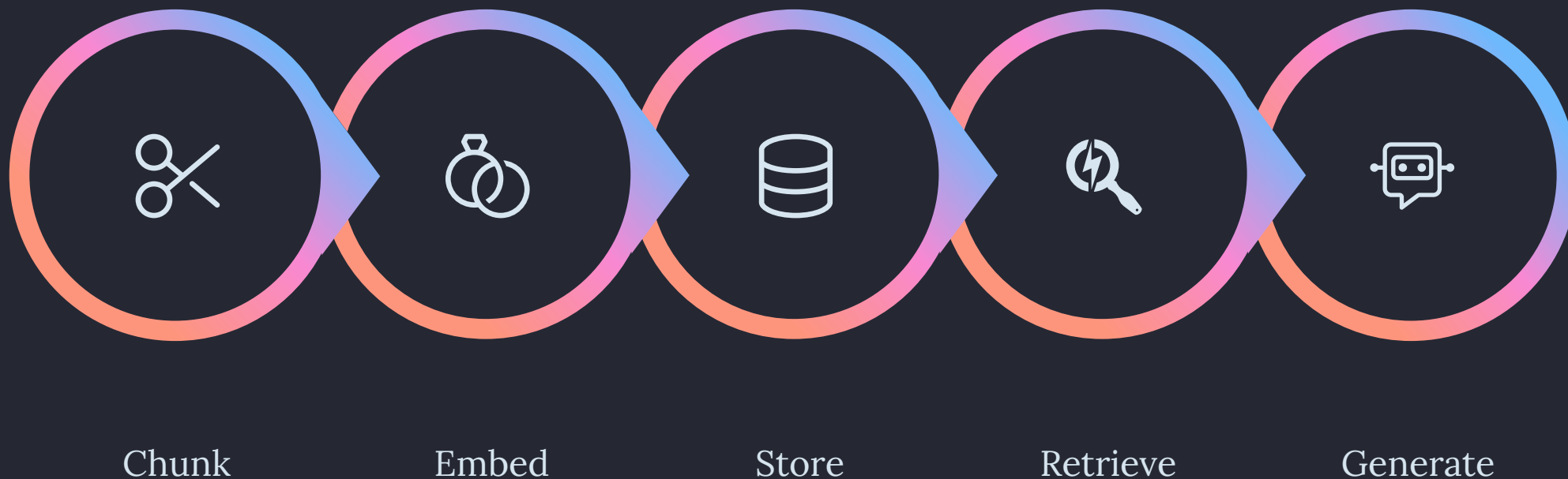


RAG ARCHITECTURE SERIES

# Naive RAG

The simplest Retrieval-Augmented Generation approach — text embeddings, a vector database, and an LLM. Fast to implement and effective for clean, extractable text from PDFs, documents, and web pages.

# How Naive RAG Works



Each document is chunked and embedded, then stored in a vector database. At query time, semantic similarity search retrieves the top-k chunks, which are passed to the LLM for answer generation.

# Limitations of Naive RAG

## Text-Only by Design

Cannot process images, charts, or tables — multimodal content is ignored or lost during ingestion.

## Structure Loss

Tables and structured data are flattened into plain text, destroying relational context.

## Approximate Retrieval

Semantic similarity alone may surface irrelevant chunks, especially across multiple documents.

## Weak Multi-Hop Reasoning

Struggles when answers require synthesizing information across several sources or reasoning steps.



RAG ARCHITECTURE SERIES

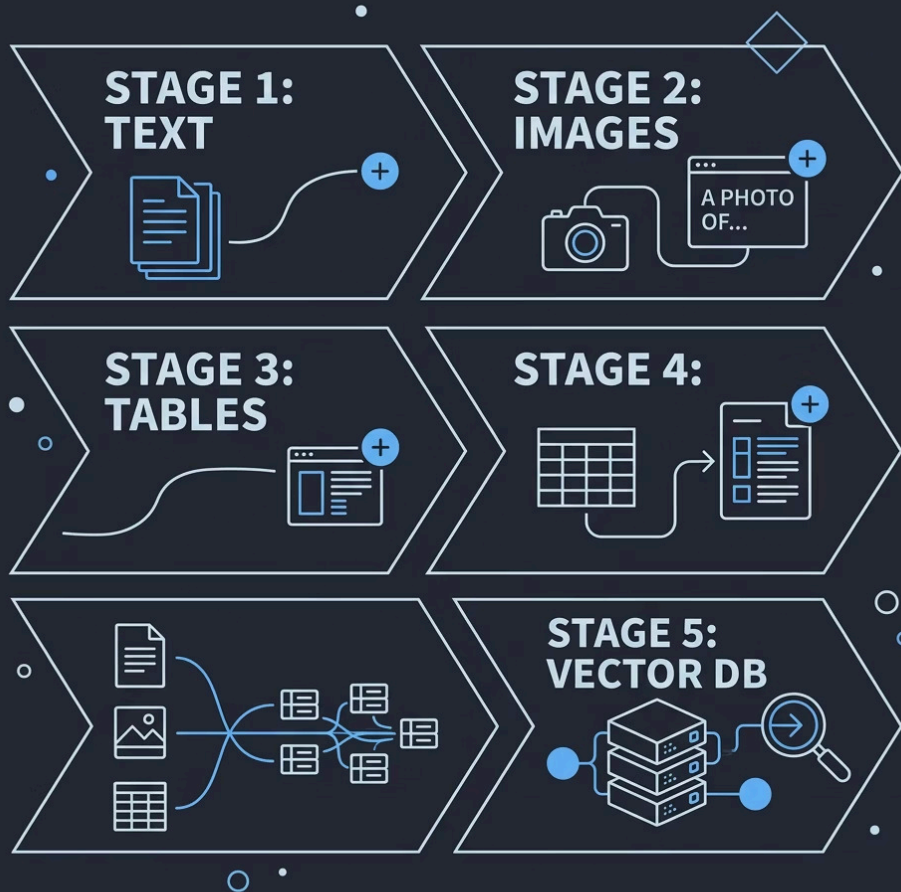
# Multimodal RAG

Extends RAG beyond plain text — combining vision and NLP models to handle images, tables, charts, and documents in a unified retrieval pipeline. Built for real-world, complex PDFs.



Made with GAMMA

# How Multimodal RAG Works



## Unified Embedding Space

Each modality is processed through a dedicated model — vision models caption images, parsers handle tables — then all outputs are embedded into a shared vector space for unified retrieval.

- ❏ Preprocessing quality is critical: OCR errors and poor captions directly degrade retrieval accuracy downstream.

# Limitations of Multimodal RAG

## Complex Pipeline

Requires OCR, parsing, and captioning — each step adds a potential point of failure.

## Higher Latency & Cost

Multiple model calls per document increase inference time and operational cost significantly.

## Modality Alignment

Embeddings across text, images, and tables are hard to align in a single semantic space.

## Noisy Retrieval

Still returns redundant or irrelevant chunks; debugging cross-modality failures is non-trivial.

# Why We Need Reranking

Even Multimodal RAG retrieves only **approximate** matches. Top-k results may include irrelevant chunks, and the best answer is not always ranked first — a problem that compounds across large document sets.

## → Top-k is not top-relevant

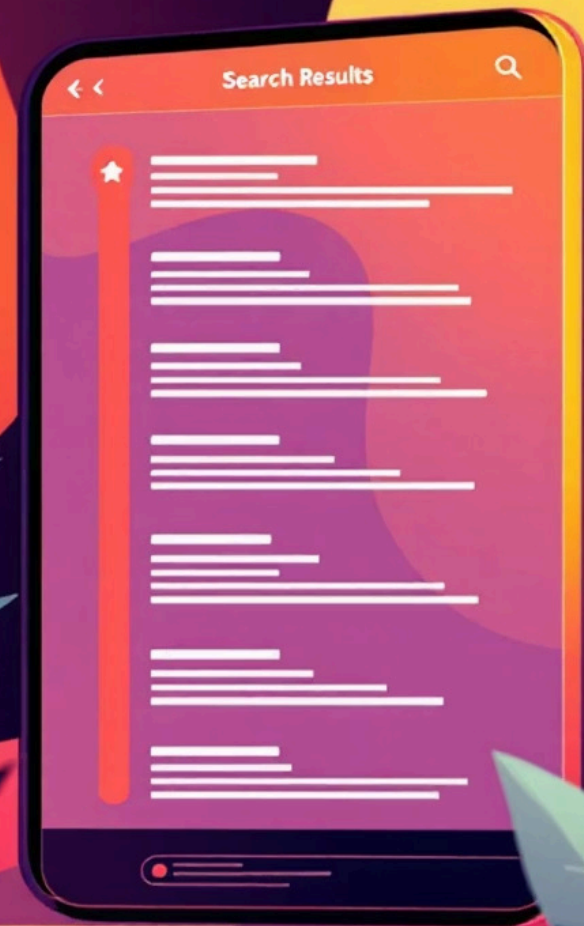
Vector similarity does not guarantee query–context relevance.

## → Noise increases with scale

More documents means more irrelevant chunks competing for the top-k slots.

## → LLM receives suboptimal context

Garbage-in, garbage-out — poor retrieval limits generation quality regardless of LLM strength.





# Reranking RAG: Overview



## A Second Stage of Precision

Reranking RAG adds a **cross-encoder reranker model** after the initial vector search. Instead of relying solely on approximate similarity, the reranker scores each candidate chunk against the query for true semantic relevance.

### Stage 1

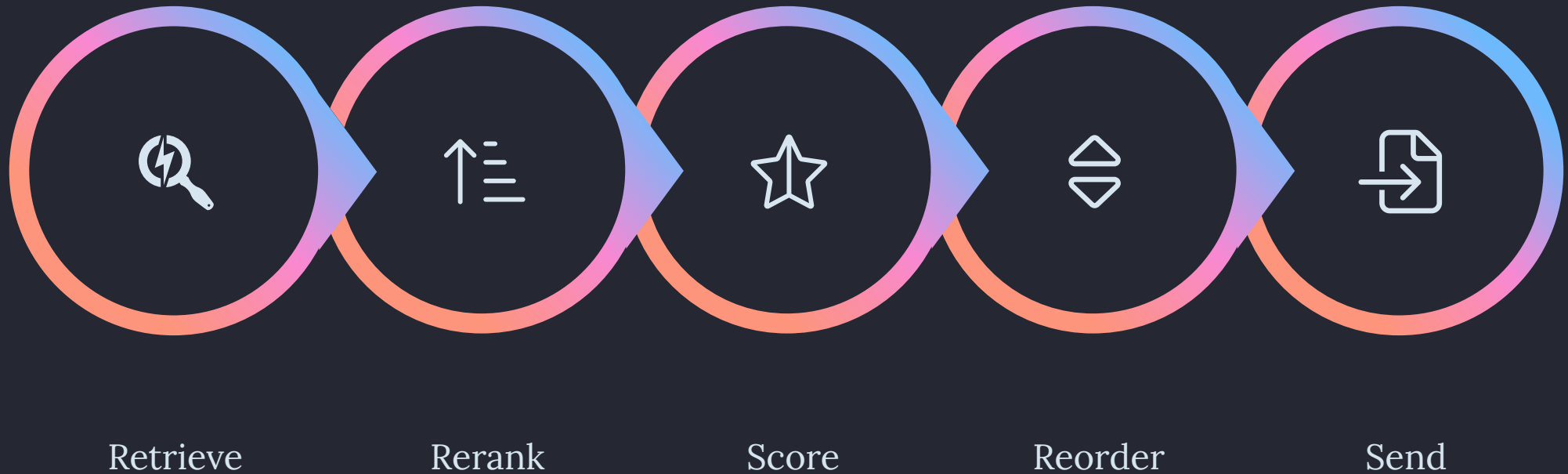
Fast vector search retrieves top-k candidates

### Stage 2

Cross-encoder reranker scores and reorders by relevance



# How Reranking RAG Works



The reranker acts as a precision filter — narrowing a broad candidate set down to the most relevant context before the LLM ever sees it, significantly improving answer quality.

# Limitations & Trade-offs

## Added Latency

A second inference pass increases end-to-end response time, especially at scale.

## Higher Cost

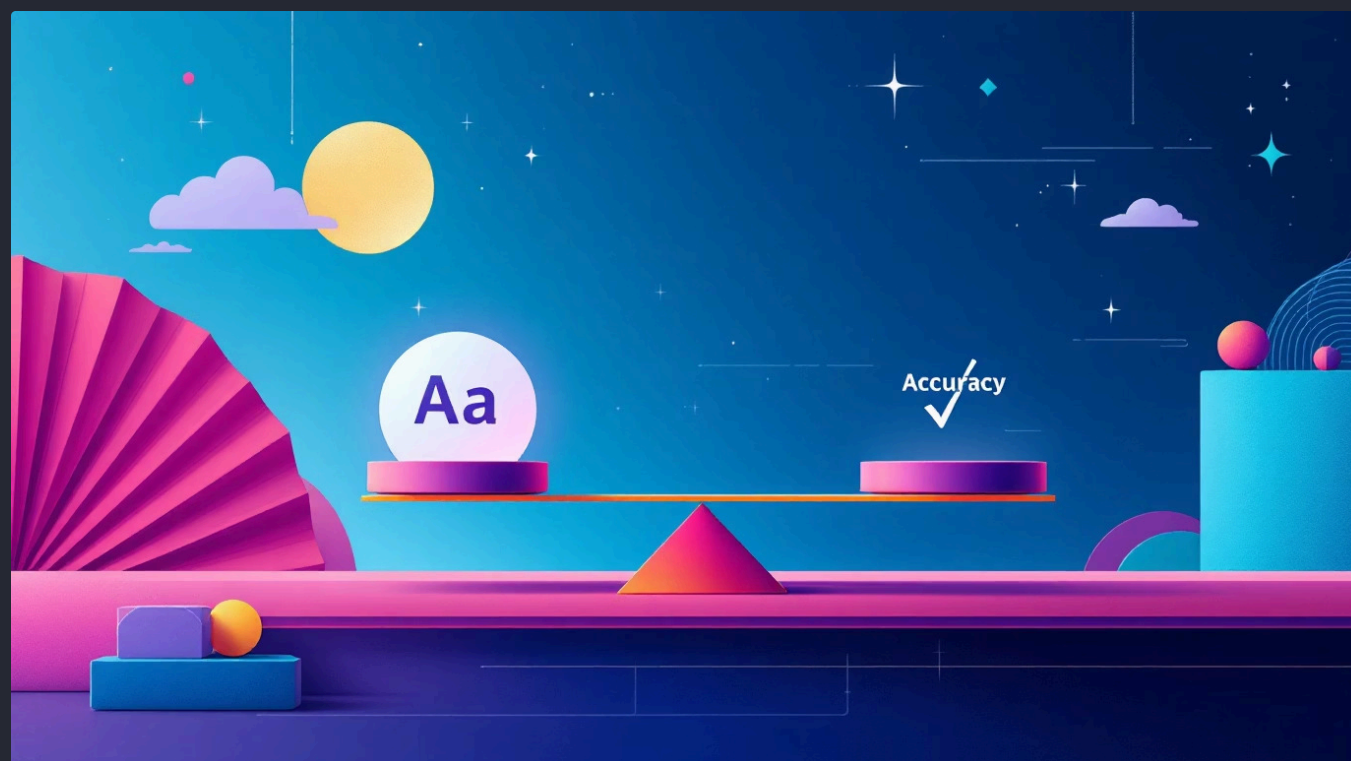
Additional model calls per query raise operational costs.

## Tuning Complexity

Parameters like **k**, **n**, and scoring thresholds require careful calibration per use case.

## Cannot Recover Missing Info

If the right chunk was never retrieved in stage 1, the reranker cannot recover it.



📌 **Key Insight:** Reranking improves precision but cannot fix upstream retrieval gaps. Invest in quality chunking and embedding first.